

RISC-V Processor Verification Documentation

1. Introduction

This document describes the verification and simulation of a simple single-cycle RISC-V processor implemented using Verilog HDL.

The processor consists of several main modules, including:

- Program Counter (PC)
- Instruction Memory
- Register File
- ALU
- Sign Extension Unit
- Control Unit
- Data Memory

A dedicated testbench was developed to verify the functionality of the processor by executing several RISC-V instructions and checking the values stored in the register file.

The main objective of the verification process is to make sure that the processor correctly fetches, decodes, executes, and writes back the results of different instructions.

2. Processor Structure

The top-level module is `top_risc_v`.

The processor connects the following modules:

1. `pc_32`
2. `mem_instr`
3. `reg_file`
4. `alu`
5. `sign_ext`
6. `control_unit`
7. `mem_data`

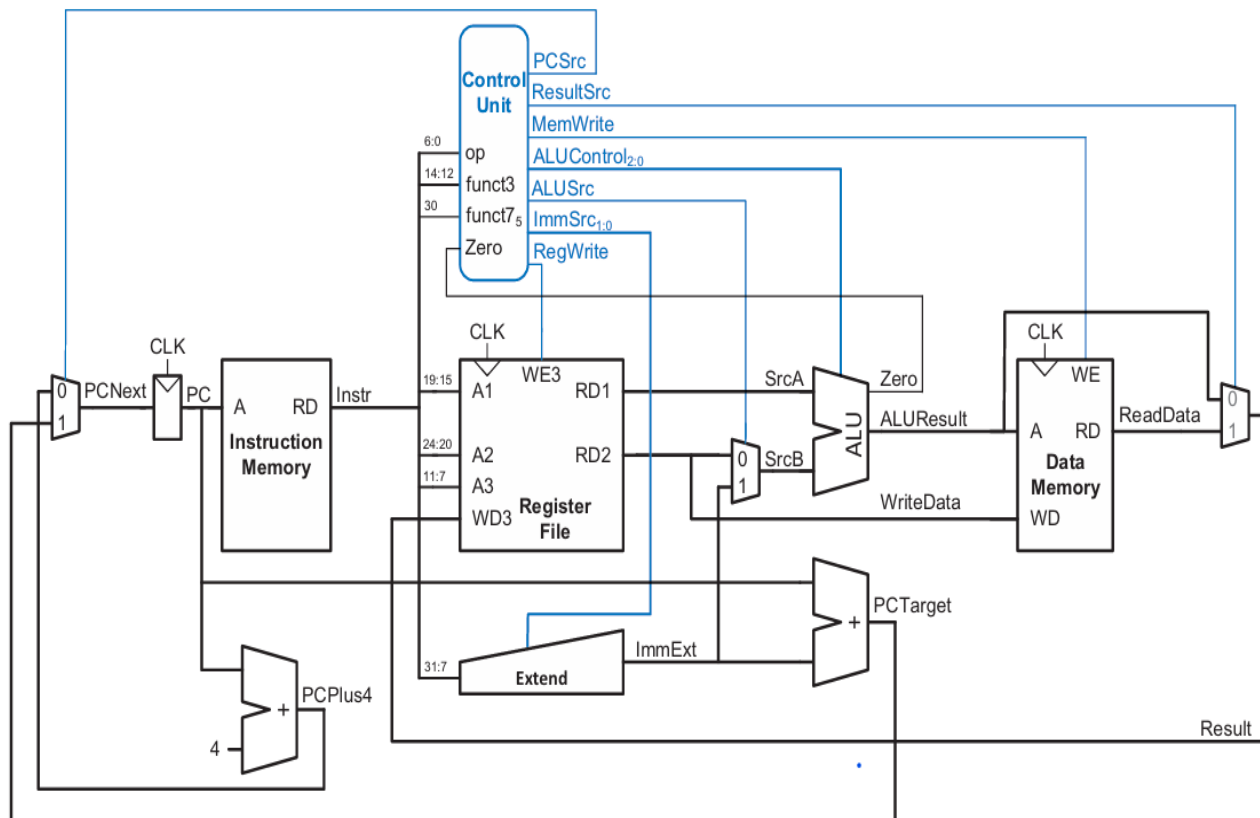
The general data flow is:

Program Counter → Instruction Memory → Control Unit / Register File → ALU → Data Memory / Register File

The ALU result is also connected to the top-level output:

```
assign alu_result = w_alu_result;
```

Figure 1: RISC-V Processor Block Diagram



single-cycle processor

3. Instruction Memory

The instruction memory contains a sequence of RISC-V instructions used for verification.

The following instructions were stored in the instruction memory:

Address	Instruction	Expected Result
0	ADDI x1, x0, 5	x1 = 5
4	ADDI x2, x0, 3	x2 = 3
8	ADD x3, x1, x2	x3 = 8
12	SUB x4, x1, x2	x4 = 2
16	AND x5, x1, x2	x5 = 1
20	OR x6, x1, x2	x6 = 7
24	XOR x7, x1, x2	x7 = 6
28	SLT x8, x1, x2	x8 = 0
32	ADDI x5, x0, -1	x5 = -1
36	SW x1, 4(x0)	Memory[1] = 5
40	LW x6, 4(x0)	x6 = 5

The instruction memory uses word-aligned addressing. Therefore, the instruction address is divided by four using:

```
m_instr [in_instr_addr >> 2]
```

This allows the processor to access the instructions sequentially at addresses 0, 4, 8, 12, and so on.

4. Testbench

A testbench named tb_top_risc_v was created to verify the processor.

The testbench generates the clock signal using:

```
always #5 clk = ~clk;
```

Therefore, the clock period is 10 ns.

The processor is initially held in reset:

```
rst = 1;
```

After 10 ns, reset is released:

```
#10;
```

```
rst = 0;
```

The processor then starts fetching and executing instructions from the instruction memory.

5. Verification Task

A simple task called check was created to compare the expected value with the actual value stored in a register.

The task is:

```
task check;
```

```
    input [31:0] expected;
```

```
    input [31:0] actual;
```

```
    begin
```

```
        if (expected == actual)
```

```
            $display("pass");
```

```
        else
```

```
            $display("fail: expected = %d, actual = %d",
```

```
                    expected , actual);
```

```
    end
```

```
endtask
```

The same task is reused multiple times to test different instructions.

For example:

```
check(5, top_uut.reg_file_c.rf[1]);
```

checks that register x1 contains the expected value 5.

Another example is:

```
check(8, top_uut.reg_file_c.rf[3]);
```

which checks the result of:

```
ADD x3, x1, x2
```

The expected result is:

$5 + 3 = 8$

Using one task multiple times makes the testbench simple and avoids repeating the same comparison code.

6. Test Cases

The following test cases were executed.

Test 1 – ADDI

Instruction:

```
ADDI x1, x0, 5
```

Expected result:

$x1 = 5$

Testbench check:

```
check(5, top_uut.reg_file_c.rf[1]);
```

Test 2 – ADDI

Instruction:

ADDI x2, x0, 3

Expected result:

x2 = 3

Testbench check:

```
check(3, top_uut.reg_file_c.rf[2]);
```

Test 3 – ADD

Instruction:

ADD x3, x1, x2

Since:

x1 = 5

x2 = 3

the expected result is:

x3 = 8

Testbench check:

```
check(8, top_uut.reg_file_c.rf[3]);
```

Test 4 – SUB

Instruction:

SUB x4, x1, x2

Expected result:

x4 = 5 - 3 = 2

Testbench check:

```
check(2, top_uut.reg_file_c.rf[4]);
```

Test 5 – AND

Instruction:

AND x5, x1, x2

The binary values are:

5 = 0101

3 = 0011

Therefore:

0101 AND 0011 = 0001

Expected result:

x5 = 1

Test 6 – OR

Instruction:

OR x6, x1, x2

The expected result is:

5 OR 3 = 7

Therefore:

x6 = 7

Test 7 – XOR

Instruction:

XOR x7, x1, x2

The expected result is:

5 XOR 3 = 6

Initially, this test failed because the control_unit did not contain the XOR decoding case.

The missing case was:

`3'b100 : alu_control = 3'b100;`

After adding this case, the control unit correctly selects the XOR operation in the ALU.

Test 8 – SLT

Instruction:

`SLT x8, x1, x2`

The processor compares:

$5 < 3$

The condition is false, therefore:

`x8 = 0`

Test 9 – Negative Immediate

Instruction:

`ADDI x5, x0, -1`

The expected mathematical result is:

`x5 = -1`

In a 32-bit unsigned display, the same value appears as:

`4294967295`

which is:

`32'hFFFFFFFF`

Therefore, the result is correct.

Test 10 – Store Word

Instruction:

SW x1, 4(x0)

Since:

$x1 = 5$

the processor should store the value 5 at memory address 4.

The data memory uses word addressing:

$m_data[addr \gg 2]$

Therefore:

$4 \gg 2 = 1$

So the expected result is:

$m_data[1] = 5$

Test 11 – Load Word

Instruction:

LW x6, 4(x0)

The processor reads the value previously stored by the SW instruction.

Expected result:

$x6 = 5$

This test verifies both the data memory read operation and the write-back path to the register file.

7. Simulation Output

During simulation, the testbench prints the current processor state using \$monitor.

The monitored signals include:

- Simulation time
- Reset
- Program Counter
- Current instruction
- Register read data
- Immediate value
- ALU result
- Values of registers x1 through x8

Example output:

```
time = 15 | rst=0 | pc=00000004 | instr=00300113 | rd1=0 | rd2=0 |  
imm=3 | alu_out=3 | x1=5 | x2=0 | x3=0
```

This output shows that after the first instruction is executed, register x1 contains the expected value 5.

For the ADD instruction:

```
time = 25 | rst=0 | pc=00000008 | instr=002081b3 | rd1=5 | rd2=3 | imm=2 |  
alu_out=8 | x1=5 | x2=3 | x3=0
```

The ALU receives:

```
src_a = 5  
src_b = 3
```

and produces:

```
alu_out = 8
```

After the clock edge, the result is written to x3.

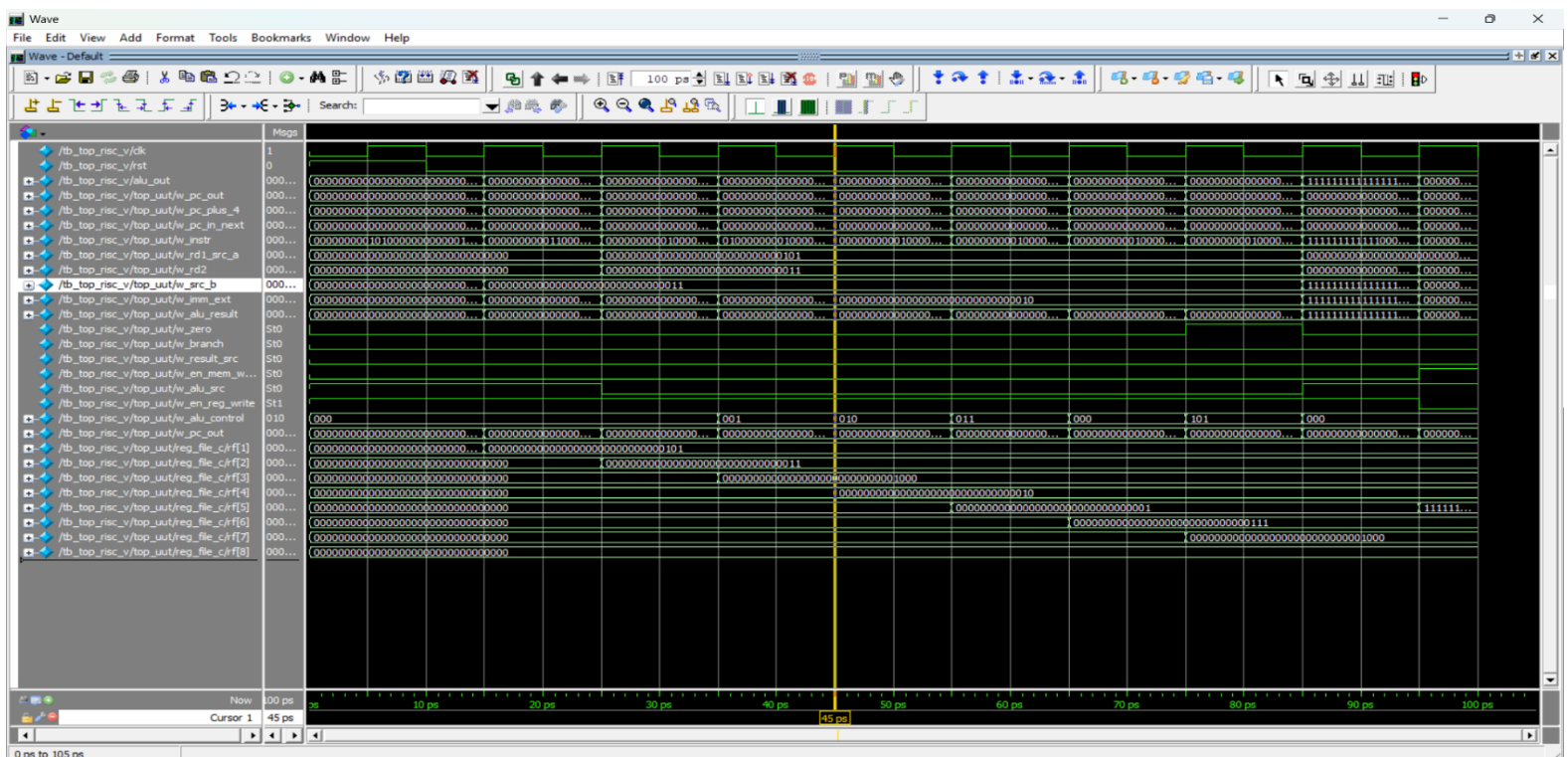
8. Waveform Verification

The ModelSim/Questa waveform was used to observe the internal operation of the processor.

The following signals are recommended to be displayed in the waveform:

Clock and Reset	Program Counter	Control Signals	Instruction	Register File	Immediate and ALU	Data Memory
clk	w_pc_out	w_branch	w_instr	w_rd1_src_a	w_imm_ext	w_read_data
rst	w_pc_plus_4	w_result_src		w_rd2	w_src_b	m_data[1]
	w_pc_target	w_en_mem_write		w_result	w_alu_result	
	w_pc_in_next	w_alu_src		rf[1] rf[2]	w_zero	
		w_en_reg_write		rf[3] rf[4] rf[5]		
		w_imm_src		rf[6]		
		w_alu_control		rf[7]		

Figure 2: RISC-V Processor Waveform During Simulation



These signals allow the complete execution path of an instruction to be observed from instruction fetch to the final register or memory update.

9. Detected Issue and Correction

During verification, the XOR instruction initially failed.

The expected result was:

5 XOR 3 = 6

However, the simulation produced:

expected = 6

actual = 0

The ALU already supported XOR:

3'b100: alu_result = src_a ^ src_b;

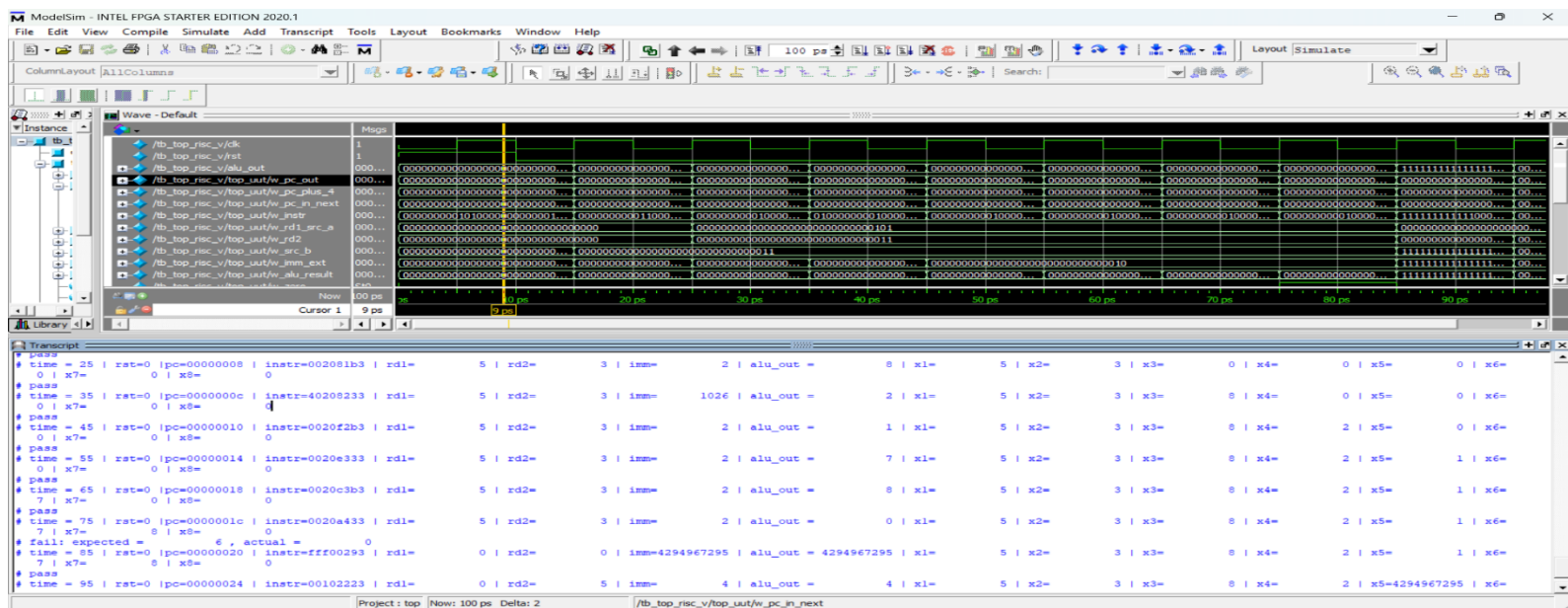
However, the control unit did not generate the corresponding ALU control signal for the XOR instruction.

The missing control case was added:

3'b100 : alu_control = 3'b100;

After this correction, the control unit correctly selects the XOR operation.

This demonstrates that the testbench was able to detect an actual functional issue in the processor design.



10. Notes About the Immediate Value

During simulation, some R-type instructions showed non-zero values on the imm signal.

For example, an ADD or SUB instruction may show a value on w_imm_ext.

This does not affect the execution of R-type instructions because the control signal: w_alu_src = 0

selects the second register value rather than the immediate value.

Therefore, for R-type instructions, the ALU uses:

src_a = rd1

src_b = rd2

and the immediate value is not used.

11. Verification Summary

The processor was tested using multiple instruction types:

Instruction Type	Instruction Verification	
Immediate	ADDI	PASS
Arithmetic	ADD	PASS
Arithmetic	SUB	PASS
Logical	AND	PASS
Logical	OR	PASS
Logical	XOR	Corrected and verified
Comparison	SLT	PASS
Negative Immediate	ADDI -1	PASS
Memory Write	SW	PASS
Memory Read	LW	PASS

The testbench successfully verified the main datapath components and detected a missing XOR control case.

```

add wave -position insertpoint sim:/tb_top_risc_v/*
VSI5> run
# time = 0 | rst=1 | pc=00000000 | instr=00500093 | rd1= 0 | rd2= 0 | imm= 5 | alu_out = 5 | x1= 0 | x2= 0 | x3= 0 | x4= 0 | x5= 0 | x6=
0 | x7= 0 | x8= 0
# time = 10 | rst=0 | pc=00000000 | instr=00500093 | rd1= 0 | rd2= 0 | imm= 5 | alu_out = 5 | x1= 0 | x2= 0 | x3= 0 | x4= 0 | x5= 0 | x6=
0 | x7= 0 | x8= 0
# time = 15 | rst=0 | pc=00000004 | instr=00300113 | rd1= 0 | rd2= 0 | imm= 3 | alu_out = 3 | x1= 5 | x2= 0 | x3= 0 | x4= 0 | x5= 0 | x6=
0 | x7= 0 | x8= 0
# pass
# time = 25 | rst=0 | pc=00000008 | instr=002081b3 | rd1= 5 | rd2= 3 | imm= 2 | alu_out = 8 | x1= 5 | x2= 3 | x3= 0 | x4= 0 | x5= 0 | x6=
0 | x7= 0 | x8= 0
# pass
# time = 35 | rst=0 | pc=0000000c | instr=40208233 | rd1= 5 | rd2= 3 | imm= 1026 | alu_out = 2 | x1= 5 | x2= 3 | x3= 8 | x4= 0 | x5= 0 | x6=
0 | x7= 0 | x8= 0
# pass
# time = 45 | rst=0 | pc=00000010 | instr=0020f2b3 | rd1= 5 | rd2= 3 | imm= 2 | alu_out = 1 | x1= 5 | x2= 3 | x3= 8 | x4= 2 | x5= 0 | x6=
0 | x7= 0 | x8= 0
# pass
# time = 55 | rst=0 | pc=00000014 | instr=0020e333 | rd1= 5 | rd2= 3 | imm= 2 | alu_out = 7 | x1= 5 | x2= 3 | x3= 8 | x4= 2 | x5= 1 | x6=
0 | x7= 0 | x8= 0
# pass
# time = 65 | rst=0 | pc=00000018 | instr=0020c3b3 | rd1= 5 | rd2= 3 | imm= 2 | alu_out = 8 | x1= 5 | x2= 3 | x3= 8 | x4= 2 | x5= 1 | x6=
7 | x7= 0 | x8= 0
# pass
# time = 75 | rst=0 | pc=0000001c | instr=0020a433 | rd1= 5 | rd2= 3 | imm= 2 | alu_out = 0 | x1= 5 | x2= 3 | x3= 8 | x4= 2 | x5= 1 | x6=
7 | x7= 8 | x8= 0
# fail: expected = 6, actual = 0
# time = 85 | rst=0 | pc=00000020 | instr=fff00293 | rd1= 0 | rd2= 0 | imm=4294967295 | alu_out = 4294967295 | x1= 5 | x2= 3 | x3= 8 | x4= 2 | x5= 1 | x6=
7 | x7= 8 | x8= 0
# pass
# time = 95 | rst=0 | pc=00000024 | instr=00102223 | rd1= 0 | rd2= 5 | imm= 4 | alu_out = 4 | x1= 5 | x2= 3 | x3= 8 | x4= 2 | x5=4294967295 | x6=
7 | x7= 8 | x8= 0
# pass

```

12. Conclusion

The implemented RISC-V processor was successfully simulated and verified using a dedicated Verilog testbench.

The verification process tested arithmetic, logical, comparison, immediate, and memory instructions.

The testbench used a simple reusable task to compare expected and actual register values. Multiple tests were performed using the same task, which kept the testbench simple and easy to understand.

The simulation waveform and console output were also used to observe the internal processor signals, including the program counter, instruction, register values, immediate value, control signals, and ALU result.

During verification, an issue was detected in the XOR instruction decoding. The issue was fixed by adding the missing XOR case to the control unit.

After the correction, the processor was able to correctly execute the tested instructions and produce the expected results.

Overall, the verification process confirmed the correct operation of the main components of the RISC-V processor and demonstrated that the testbench can detect functional errors in the design.